Лабораторная работа 4

Базовые растровые алгоритмы (100 баллов).

Выполнение настоящей работы имеет целью закрепление теоретического материала и практическое освоение основных возможностей по:

- использованию базовых алгоритмов растеризации отрезков и кривых:
  - пошаговый алгоритм;
  - алгоритм ЦДА;
  - алгоритм Брезенхема;
  - алгоритм Брезенхема (окружность).

Задача:

- Написать приложение/веб-приложение, иллюстрирующее работу базовых растровых алгоритмов (4 алгоритма).
- На проверку сдаются: exe, который должен работать на любом ПК под Windows XP /веб-приложение, выложенное в общий доступ; исходный код; сопроводительная документация.

Баллы:

- Корректность работы методов (проиллюстрировать один пример сопроводительными вычислениями): **10+10+20+20 = 60 баллов**.
- Краткий отчет с указанием временных характеристик реализованных алгоритмов (вычислить время в своей реализации): **20 баллов**.
- Дружелюбный и удобный интерфейс (масштаб; обязателен вывод системы координат, осей, линий сетки, подписей; пояснить, каким образом целочисленные координаты привязаны к дискретной сетке): **20 баллов**.

Задания на дополнительные баллы:

- Реализация алгоритма Кастла-Питвея (найти в интернете понятное описание и разобраться с алгоритмом).
- Визуализация сглаженных линий (с пояснениями).

ДОПОЛНЕНИЯ К ЛАБОРАТОРНОЙ РАБОТЕ 4 ДЛЯ ГРУППЫ 10А

Шаг 1. Изоляция рендеринга

- **Полный запрет на использование функций рисования примитивов.**
    - Приложение должно работать с виртуальным растром — **двумерным массивом чисел/булевых флагов** (например, int[,] virtualGrid = new int[Width, Height]).
    - Алгоритмы Брезенхема, ЦДА и окружности должны **только заполнять этот числовой массив** (устанавливать флаг «пиксель закрашен»).
    - Отрисовка сетки и пикселей в UI происходит *после* работы алгоритма на основе этого массива.

Шаг 2. Алгоритмы и Оптимизация

1. Бенчмарк производительности

- Вместо замера времени одной линии, программа должна содержать модуль **Стресс-тестирования (Benchmark)**.
- **Требование:** Программа генерирует «солнце» или «веер» из **100 000 случайных отрезков** (во всех 8 октантах) и **50 000 окружностей** разного радиуса. Замеряется суммарное время генерации *чистого виртуального растра* (без учета UI) с помощью высокоточных таймеров (Stopwatch в .NET / time.perf_counter() в Python и т.п.). Необходимо построить график зависимости времени от количества примитивов и длины линий.

2. Корректная обработка всех 8 октантов (Ловушка Брезенхема)

- **Требование:** Алгоритмы Брезенхема (для линии и окружности) должны быть **обобщенными**.

3. Сглаживание и Кастл-Питвей

- **Алгоритм Ву (Wu's Anti-aliasing)**: обязателен для сглаживания линий. Необходимо реализовать честное вычисление интенсивности пикселей (дробная часть координаты определяет прозрачность Alpha двух соседних пикселей).
- **Алгоритм Кастла-Питвея (Castle-Piteway)**

целочисленный алгоритм для генерации кривых (конических сечений/дуг).

Шаг 3. Обновленная шкала оценки (10-балльная система)

Баллы	Критерии выполнения
1–3 балла	Реализованы базовые формулы из интернета (часто работают только в 1-2 октантах). Вместо дискретной сетки используется обычный холст. Замеры времени сделаны «на глаз» для одной линии.
4–6 баллов	Алгоритмы (ЦДА, Брезенхем линия/окружность) работают корректно во всех **8 октантах**. Реализована **виртуальная сетка (массив)**, интерфейс позволяет масштабировать пиксели и четко показывает привязку координат.
7–8 баллов	Реализован **автоматический бенчмарк** на 100 000+ примитивов с построением графиков производительности. Добавлен честный **алгоритм Ву** для сглаживания с расчетом прозрачности граничных пикселей.
9–10 баллов	Дополнительно реализован целочисленный алгоритм **Кастла-Питвея** для кривых. Проведен глубокий аналитический разбор в отчете: математически обосновано, почему ЦДА медленнее Брезенхема, и подтверждено это графиками бенчмарка.

🏛️ Архитектура проекта для ЛР № 4 (разделение на модули)

1. RasterEngine – библиотека статических методов, содержащая чистые алгоритмы (DDALine, BresenhamLine, WuLine, BresenhamCircle). Они принимают координаты и интерфейс/коллбэк IRasterTarget, у которого есть только один метод: SetPixel(int x, int y, double intensity).
2. VirtualGrid – класс, хранящий двумерную матрицу пикселей и отвечающий за сброс/накопление результатов бенчмарка.
3. GridPainter – компонент UI, который просто рисует увеличенные квадраты (сетку пикселей) на экране на основе данных из VirtualGrid.